KeepCoding Bootcamp Ciberseguridad

ANALISIS COMPARATIVO DE SEGURIDAD: MANUAL vs BASELINE vs CAI

RAUL AMO PICO

2026

Índice

PORTADA ..... **¡Error! Marcador no definido.**
Índice 2
1. Introducción **3**
2. Marco teórico..... **5**
3. Diseño Experimental **7**
4. Fase 1 – Análisis Manual (Ground Truth) **8**
5. Fase 2 – Baseline Automatizado **18**
6. Fase 3 – Análisis Asistido por Inteligencia Artificial (CAI) **29**
7. Análisis Comparativo y Estudio Estadístico **30**
8. Discusión final, limitaciones y conclusiones **35**

ANEXOS **39**

1. Introducción

1.1 Contexto y motivación

La evolución del desarrollo de aplicaciones web y APIs ha incrementado significativamente la superficie de ataque en entornos digitales. En la actualidad, gran parte de los servicios empresariales y plataformas digitales se apoyan en APIs REST, microservicios y arquitecturas distribuidas.

Las auditorías de seguridad tradicionales combinan análisis manual con herramientas automatizadas de escaneo. Sin embargo, en la práctica se observa una discrepancia entre la cantidad de vulnerabilidades técnicas detectadas automáticamente (por ejemplo, CVE en dependencias) y la identificación de vulnerabilidades lógicas o de autorización, que suelen requerir intervención humana.

En este contexto, surge la necesidad de evaluar si nuevas aproximaciones, como el uso de inteligencia artificial, pueden reducir esta brecha.

1.2 Problema de investigación

El problema central de este trabajo es determinar si la inteligencia artificial puede mejorar la detección de vulnerabilidades lógicas en comparación con herramientas automatizadas tradicionales, sin sustituir la validación manual.

En particular, se analiza la capacidad de tres enfoques distintos:

- Análisis manual (Ground Truth).
- Baseline automatizado mediante herramientas de escaneo.
- Análisis asistido por inteligencia artificial (CAI).

La cuestión fundamental es:

> ¿Puede la IA reducir los falsos negativos en vulnerabilidades lógicas frente a un baseline automatizado convencional?

1.3 Objetivos del trabajo

Objetivo general

Evaluar comparativamente la efectividad de análisis manual, baseline automatizado y análisis asistido por IA en la detección de vulnerabilidades en aplicaciones web y APIs.

Objetivos específicos

- Desplegar un entorno de laboratorio reproducible y aislado.
- Identificar y explotar manualmente vulnerabilidades reales (Ground Truth).
- Ejecutar herramientas automatizadas y analizar sus resultados.
- Aplicar IA para interpretar resultados y detectar riesgos lógicos.
- Cuantificar diferencias mediante métricas estadísticas.
- Valorar un modelo híbrido de auditoría.

1.4 Alcance y limitaciones

El estudio se realiza sobre dos laboratorios deliberadamente vulnerables: OWASP Juice Shop y VAmPI.

No se pretende generalizar resultados a entornos productivos reales, sino validar experimentalmente diferencias metodológicas.

Limitaciones principales:

- Número limitado de aplicaciones analizadas.
- Entornos controlados y no productivos.
- Uso de versión “gratuita” de herramientas de IA.
- No evaluación de precisión completa por ausencia de dataset balanceado.

1.5 Estructura del documento

El documento se organiza en los siguientes capítulos principales:

- Introducción y marco teórico.
- Diseño experimental.
- Análisis manual (Ground Truth).
- Baseline automatizado.
- Análisis asistido por IA.
- Estudio estadístico comparativo.
- Discusión y conclusiones.

2. Marco Teórico

2.1 Seguridad en aplicaciones Web y APIs

Las aplicaciones web y APIs constituyen uno de los principales vectores de ataque actuales. A diferencia de vulnerabilidades puramente técnicas, muchas brechas de seguridad derivan de errores de diseño en control de acceso y validación de datos.

2.2 OWASP Top 10 (Web)

OWASP identifica las vulnerabilidades más críticas en aplicaciones web, entre ellas:

- Broken Access Control
- Injection
- Security Misconfiguration
- Insecure Design

Este marco se utilizó como referencia para clasificar vulnerabilidades detectadas en Juice Shop.

2.3 OWASP API Security Top 10

En el ámbito de APIs, OWASP destaca vulnerabilidades como:

- Broken Object Level Authorization (BOLA)
- Broken Object Property Level Authorization (BOPLA)
- Broken Function Level Authorization
- Security Misconfiguration

Este estándar fue utilizado para clasificar vulnerabilidades identificadas en VAmPI.

2.4 Vulnerabilidades lógicas vs técnicas

Las vulnerabilidades técnicas suelen estar asociadas a errores conocidos en software o dependencias (CVE).

Las vulnerabilidades lógicas, en cambio, derivan de errores en el diseño de autorización o validación contextual, y son más difíciles de detectar mediante herramientas automatizadas.

2.5 Automatización en auditoría

Herramientas como OWASP ZAP y Trivy permiten detectar vulnerabilidades técnicas y configuraciones inseguras.

No obstante, su capacidad para identificar errores lógicos es limitada.

2.6 Inteligencia Artificial aplicada a ciberseguridad

La IA puede analizar contratos OpenAPI, identificar patrones de riesgo y priorizar hallazgos.

Su potencial radica en la interpretación contextual, aunque no sustituye la explotación manual.

3. Diseño Experimental

3.1 Metodología

Se adoptó una metodología comparativa experimental en tres fases:

1. Explotación manual.
2. Escaneo automatizado.
3. Análisis asistido por IA.

Los tres enfoques se aplicaron sobre los mismos entornos para garantizar comparabilidad.

3.2 Entorno de laboratorio

3.2.1 Infraestructura técnica

- Windows como host.
- Docker para despliegue de laboratorios.
- Kali Linux como máquina atacante.
- VirtualBox con doble interfaz (NAT + Host-Only).

3.2.2 Arquitectura de red

Kali se configuró con doble interfaz:

- NAT para acceso a Internet.
- Host-Only (192.168.56.0/24) para comunicación aislada.

Los servicios Docker fueron publicados mediante port binding y redirigidos a la interfaz Host-Only mediante portproxy.

3.2.3 Despliegue de laboratorios

Se desplegaron:

- OWASP Juice Shop (aplicación web).
- VAmPI (API vulnerable).

Ambos accesibles desde Kali en:

- http://192.168.56.1:3000
- http://192.168.56.1:5002

4. Fase 1 – Análisis Manual (Ground Truth)

4.1 Metodología de explotación

La fase de Ground Truth constituyó el núcleo experimental del estudio. El objetivo fue identificar, explotar y validar empíricamente vulnerabilidades reales en los laboratorios VAmPI y OWASP Juice Shop.

El enfoque aplicado fue manual y semiautomatizado, priorizando:

- Validación práctica de explotabilidad.
- Confirmación de impacto real.
- Documentación reproducible de evidencias.
- Pruebas positivas y negativas.
- Encadenamiento de vulnerabilidades.

A diferencia de un simple escaneo automatizado, esta fase simuló un escenario real de auditoría profesional, en el que el analista interpreta respuestas, modifica parámetros y valida persistencia del impacto. Ver también el informe completo en la carpeta 02_Analisis_Manual_GroundTruth.

Las evidencias generadas se almacenaron sistemáticamente en el directorio:

`\02_Analisis_Manual_GroundTruth\`

4.2 Vulnerabilidades detectadas en VAmPI

El análisis de la API VAmPI permitió confirmar vulnerabilidades correspondientes a todas las categorías del OWASP API Security Top 10.

Resumen global

Se identificaron y validaron 10 vulnerabilidades, todas explotables o verificables mediante pruebas reproducibles.

ID	Categoría OWASP API	Estado
API-01	Broken Object Level Authorization	Confirmada
API-02	Broken Authentication	Confirmada
API-03	Broken Object Property Level Authorization	Confirmada
API-04	Unrestricted Resource Consumption	Confirmada
API-05	Broken Function Level Authorization	Confirmada
API-06	Mass Assignment	Confirmada
API-07	Security Misconfiguration	Confirmada
API-08	Lack of Rate Limiting	Confirmada
API-09	Improper Session Management	Confirmada
API-10	Excessive Data Exposure	Confirmada

4.2.1 Vulnerabilidades críticas de autorización

Se identificaron múltiples fallos de autorización:

- **BOLA (API-01)**: acceso a datos de otros usuarios mediante manipulación del identificador.
- **BOPLA (API-03)**: modificación de propiedades sensibles como contraseña o correo electrónico.
- **Broken Function Level Authorization (API-05)**: acceso a funciones administrativas sin privilegios adecuados.

Estos fallos permitieron acceso indebido a recursos de terceros y modificación persistente de cuentas.

Impacto:

- Toma de control de cuentas.
- Elevación de privilegios.
- Compromiso sistémico del sistema.

4.2.2 Fallos de autenticación y sesión

Se confirmó la existencia de:

- Exposición de credenciales en texto plano mediante endpoint de depuración.
- Gestión deficiente de tokens JWT.
- Ausencia de revocación tras cambios críticos.
- Tokens válidos incluso tras modificación de credenciales.

Impacto:

- Persistencia de sesión comprometida.
- Compromiso completo del modelo de autenticación.

4.2.3 Fallos de diseño y configuración

Se identificaron además:

- Endpoint administrativo sin autenticación (/createdb).
- Ausencia de rate limiting en login.
- Aceptación de objetos JSON sin validación estricta (Mass Assignment).
- Respuestas excesivamente detalladas.

Estos fallos evidencian debilidad estructural en el diseño de la API.

4.2.4 Encadenamiento de vulnerabilidades

Uno de los hallazgos más relevantes fue la capacidad de encadenar múltiples vulnerabilidades:

1. Enumeración de usuarios.
2. Obtención de credenciales.
3. Autenticación como administrador.
4. Modificación persistente de cuentas.
5. Persistencia de sesión.

Este encadenamiento demuestra que el impacto no es aislado, sino sistémico.

4.2.5 Tabla resumen de vulnerabilidades – VAmPI

ID	Endpoint	Vulnerabilidad	OWASP API Top 10	Severidad
API-01	/users/v1/{username}	Broken Object Level Authorization (BOLA)	API1	Alta
API-02	/users/v1/_debug	Exposición de credenciales en texto plano	API2 / API8	Crítica
API-03	/users/v1/{username}/password	Broken Object Property Level Authorization (BOPLA)	API3	Alta
API-04	/createdb	Función administrativa sin autenticación	API5	Alta
API-05	/users/v1/{username}	Broken Function Level Authorization	API5	Alta
API-06	/users/v1/{username}/email	Broken Object Property Level Authorization (BOPLA)	API3	Alta
API-07	/users/v1/_debug	Security Misconfiguration	API7	Alta
API-08	/users/v1/login	Ausencia de Rate Limiting	API4	Media/Alta
API-09	JWT	Gestión inadecuada de sesión	API9	Media
API-10	/users/v1	Excessive Data Exposure	API10	Media

4.3 Vulnerabilidades detectadas en OWASP Juice Shop

El análisis manual de la aplicación web permitió identificar 9 vulnerabilidades explotables y 1 prueba negativa correctamente documentada.

4.3.1 Vulnerabilidades de acceso y exposición

- Forced Browsing en directorio /ftp.
- File Disclosure mediante acceso directo a archivos internos.
- Directory Listing activo.
- Excessive Data Exposure en búsquedas.

Impacto:

- Exposición de información sensible.
- Enumeración sistemática de recursos.

4.3.2 Vulnerabilidades de inyección

Se confirmaron:

- Reflected XSS.
- Stored XSS.
- DOM-based XSS.

Estos ataques permiten ejecución de código arbitrario en el navegador del usuario.

4.3.3 Fallos de configuración

Se detectó ausencia de cabeceras de seguridad como:

- Content Security Policy
- HSTS
- X-Frame-Options

Esto incrementa la superficie de ataque frente a XSS y clickjacking.

4.3.4 Fallos de lógica de negocio

Se identificó un Business Logic Flaw que permitía aceptar valores inconsistentes en el carrito de compra.

Impacto:

- Manipulación de precios.
- Alteración del flujo normal de negocio.

4.3.5 Prueba negativa documentada

Se intentó explotar un IDOR autenticado manipulando identificadores de recursos. La aplicación respondió correctamente con errores 401 Unauthorized.

La documentación de esta prueba negativa refuerza el rigor metodológico del análisis.

4.3.6 Tabla resumen de vulnerabilidades – OWASP Juice Shop

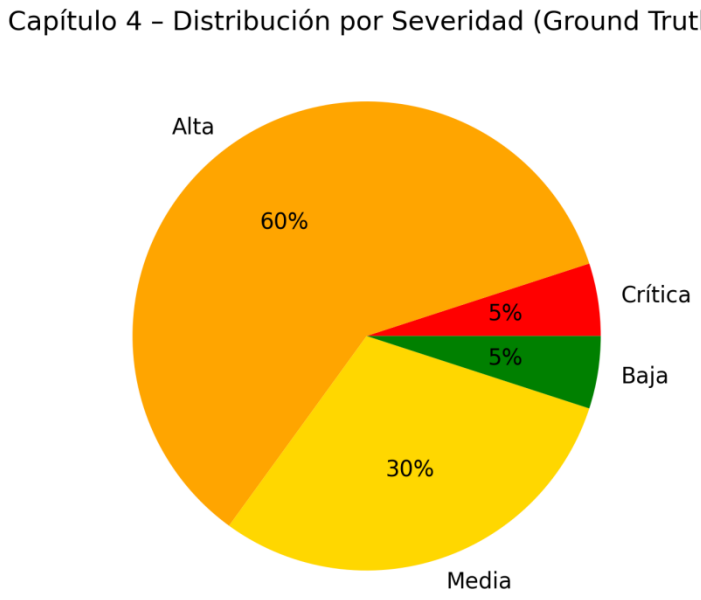
ID	Endpoint	Vulnerabilidad	OWASP Top 10	Severidad
WEB-01	/rest/products/search	Excessive Data Exposure	A01 – Broken Access Control	Media
WEB-02	/#/search	Reflected XSS	A03 – Injection	Alta
WEB-03	/api/Feedbacks	Stored XSS	A03 – Injection	Alta
WEB-04	/ftp	Forced Browsing	A01 – Broken Access Control	Alta
WEB-05	/ftp/*	File Disclosure	A01 – Broken Access Control	Alta
WEB-06	/ftp	Directory Listing	A01 – Broken Access Control	Media/Alta
WEB-07	UI (DOM)	DOM-based XSS	A03 – Injection	Alta
WEB-08	/	Security Misconfiguration	A05 – Security Misconfiguration	Media
WEB-09	/rest/basket/{id}	IDOR autenticado (prueba negativa)	A01 – Broken Access Control	Baja
WEB-10	/api/BasketItems	Business Logic Flaw	A04 – Insecure Design	Alta

4.4 Resumen cuantitativo del Ground Truth

En total se confirmaron:

- 10 vulnerabilidades en VAmPI.
- 9 vulnerabilidades explotables en Juice Shop.
- 1 prueba negativa correctamente validada en Juice Shop.

Total: 20 hallazgos documentados.



Este conjunto constituye la base de referencia para la comparación con el baseline automatizado y el análisis asistido por IA.

4.5 Conclusión del análisis manual

El análisis manual evidenció múltiples fallos críticos en control de acceso, autenticación, validación de entrada y lógica de negocio.

La explotación práctica demostró:

- Impacto real y persistente.
- Encadenamiento viable.
- Compromiso sistémico en el caso de la API.

Este conjunto de vulnerabilidades representa el “Ground Truth” frente al cual se evaluaron los métodos automatizados e IA.

A diferencia de VAmPI, donde se demostró un encadenamiento claro de vulnerabilidades con impacto sistémico, en OWASP Juice Shop los hallazgos se presentaron principalmente como vulnerabilidades independientes.

Si bien existe potencial teórico de encadenamiento (por ejemplo, mediante XSS persistente para secuestro de sesión), este estudio no desarrolló escenarios de explotación compuesta en el laboratorio web.

5. Fase 2 – Baseline Automatizado

5.1 Objetivo del Baseline

La segunda fase del estudio consistió en aplicar herramientas automatizadas de análisis de seguridad sobre los mismos laboratorios evaluados manualmente, con el fin de comparar su capacidad de detección frente al Ground Truth previamente establecido.

El objetivo no fue únicamente enumerar vulnerabilidades, sino evaluar:

- Cobertura real respecto a vulnerabilidades explotadas manualmente.
- Capacidad de detección de fallos lógicos.
- Volumen y naturaleza de hallazgos generados.
- Presencia de falsos negativos.

5.2 Herramientas utilizadas

Se emplearon las siguientes herramientas:

- **OWASP ZAP** Escaneo dinámico (DAST) de la aplicación web.
- **Trivy** Escaneo de vulnerabilidades en imágenes Docker y dependencias (SCA).

Los análisis se ejecutaron sobre:

- Imagen Docker de OWASP Juice Shop.
- Imagen Docker de VAmPI.
- Endpoints accesibles desde Kali.

Los resultados fueron exportados en formato JSON para permitir su análisis estructurado.

5.3 Resultados en OWASP Juice Shop

El escaneo de la imagen Docker de Juice Shop mediante Trivy generó un volumen elevado de vulnerabilidades asociadas a dependencias del sistema base (Debian) y paquetes Node.js.

Resumen cuantitativo

- Total vulnerabilidades detectadas (CVE): **91**
- Severidad crítica: 0
- Severidad alta: múltiples
- Severidad media y baja: predominantes

La mayoría de las vulnerabilidades correspondían a:

- Dependencias del sistema operativo base.
- Librerías Node.js incluidas en la imagen.
- Paquetes no actualizados.

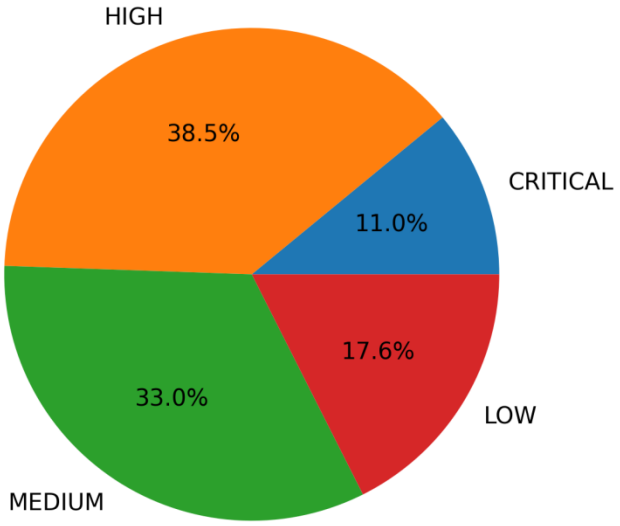
Observación clave

Ninguna de las vulnerabilidades lógicas explotadas fue identificada explícitamente como tal, directamente por las herramientas:

- Stored XSS
- Reflected XSS
- DOM-based XSS
- Business Logic Flaw
- Forced Browsing
- Excessive Data Exposure

Es decir, el baseline automatizado generó un alto volumen de CVE técnicas, pero no detectó las vulnerabilidades lógicas explotadas en la fase manual.

Capítulo 5 – Distribución de Severidad (Juice Shop)



5.4 Resultados en VAmPI

El análisis de la imagen VAmPI generó:

- Total vulnerabilidades detectadas (CVE): **12**
- Severidades principalmente medias y altas.

Las vulnerabilidades detectadas correspondían a:

- Paquetes base del sistema.
- Dependencias de entorno Python.

Observación clave

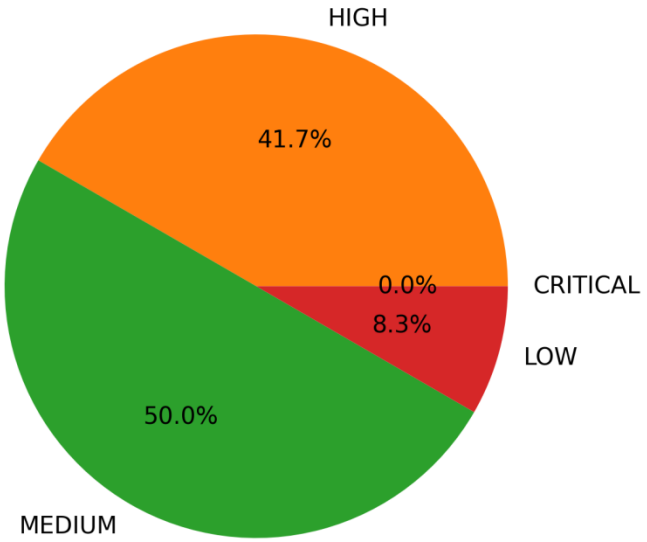
Ninguna de las siguientes vulnerabilidades críticas fue detectada por el baseline:

- BOLA
- BOPLA
- Broken Function Level Authorization

- Mass Assignment
- Falta de Rate Limiting
- Gestión deficiente de JWT
- Endpoint administrativo sin autenticación

El baseline no identificó la gran mayoría de fallos de autorización ni lógica de negocio.

Capítulo 5 – Distribución de Severidad (VAmPI)



5.5 Análisis comparativo respecto al Ground Truth

El Ground Truth estableció 20 vulnerabilidades reales:

- 10 en VAmPI.
- 10 en Juice Shop.

El baseline automatizado:

- Detectó numerosas vulnerabilidades técnicas (CVE).
- No detectó la mayoría de las vulnerabilidades lógicas explotadas manualmente.
- No identificó encadenamientos ni riesgos contextuales.

Esto evidencia una brecha significativa entre:

- Detección de superficie técnica.
- Detección de vulnerabilidades lógicas y de diseño.

5.6 Clasificación por tipo de vulnerabilidad

Los resultados del baseline pueden clasificarse en:

1) Vulnerabilidades de dependencias (CVE)

Relacionadas con paquetes del sistema o librerías.

2) Configuraciones detectables automáticamente

Algunas cabeceras HTTP faltantes identificadas por ZAP.

3) Ausencia de detección lógica

No se detectaron:

- Fallos de autorización contextual.
- Manipulación de identificadores.
- Lógica de negocio inconsistente.
- Encadenamientos.

5.7 Limitaciones observadas

Se identificaron las siguientes limitaciones del enfoque automatizado:

- Dependencia de firmas conocidas.
- Incapacidad para interpretar relaciones entre recursos.
- Falta de comprensión contextual.
- Generación de volumen elevado de hallazgos no críticos.
- Alta tasa de falsos negativos en vulnerabilidades lógicas.

5.8 Conclusión del Baseline

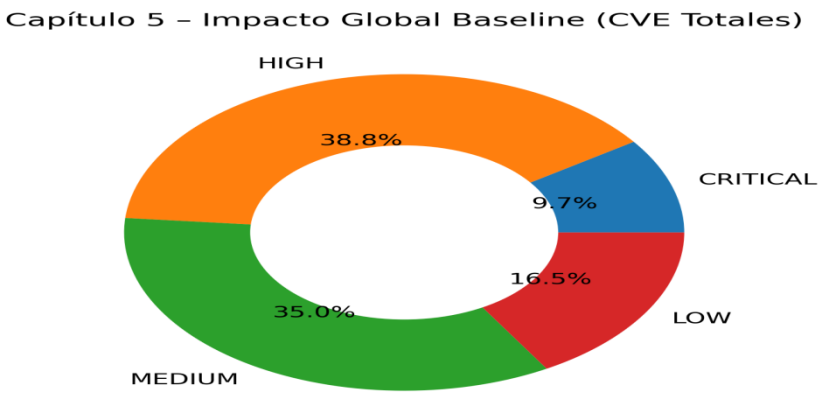
El análisis automatizado demostró eficacia en:

- Identificación de vulnerabilidades técnicas en dependencias.
- Enumeración de configuraciones inseguras.

Sin embargo, mostró limitaciones claras en:

- Detección de vulnerabilidades lógicas.
- Identificación de encadenamientos.
- Evaluación contextual del riesgo.

El baseline constituye una herramienta útil para análisis inicial, pero insuficiente como método exclusivo de auditoría.



6. Fase 3 – Análisis Asistido por Inteligencia Artificial (CAI)

6.1 Objetivo de la fase CAI

Tras la ejecución del análisis manual (Ground Truth) y del baseline automatizado, se incorporó una tercera fase basada en inteligencia artificial con el objetivo de evaluar su capacidad para:

- Interpretar resultados automatizados.
- Analizar contratos OpenAPI.
- Identificar riesgos lógicos no detectados por herramientas tradicionales.
- Reducir la tasa de falsos negativos.

El propósito no fue sustituir el análisis manual, sino evaluar si la IA puede actuar como capa intermedia interpretativa.

6.2 Metodología aplicada

La fase CAI se desarrolló en tres pasos principales:

1) Análisis del contrato OpenAPI (VAmPI)

Se proporcionó a la IA la especificación OpenAPI de la API vulnerable, permitiendo:

- Enumeración estructurada de endpoints.
- Identificación de métodos HTTP sensibles.
- Detección de endpoints administrativos.
- Evaluación de coherencia entre autenticación declarada y operaciones expuestas.

2) Interpretación de resultados del baseline

Se suministraron los resultados JSON generados por Trivy y ZAP, solicitando:

- Clasificación por criticidad real.
- Priorización contextual.
- Identificación de posibles vectores lógicos no detectados automáticamente.

3) Análisis de patrones de autorización

Se solicitó a la IA evaluar escenarios como:

- Manipulación de identificadores.
- Modificación de propiedades sensibles.
- Acceso a funciones administrativas.
- Gestión de tokens JWT.

6.3 Resultados en VAmPI

La IA identificó conceptualmente riesgos asociados a:

- Broken Object Level Authorization.
- Broken Function Level Authorization.
- Endpoints administrativos sin autenticación.
- Posibles problemas de rate limiting.
- Riesgos en gestión de JWT.

Aunque la IA no ejecutó explotación real, sí fue capaz de:

- Detectar patrones sospechosos en la definición de endpoints.
- Inferir riesgo en ausencia de controles explícitos.
- Priorizar endpoints sensibles como `/users/v1/{username}` y `/createdb`.

Resultado clave

La IA logró identificar correctamente, de forma conceptual, la mayoría de las categorías del OWASP API Top 10 presentes en el laboratorio, reduciendo significativamente la brecha respecto al Ground Truth.

6.4 Resultados en OWASP Juice Shop

En el caso de la aplicación web:

- La IA identificó posibles riesgos de XSS en endpoints dinámicos.
- Señaló exposición excesiva de datos en búsquedas.
- Detectó ausencia de cabeceras de seguridad.
- Identificó riesgo potencial en lógica de negocio.

Sin embargo, no ejecutó explotación real ni validó la persistencia.

6.5 Comparativa cuantitativa preliminar

Respecto al Ground Truth (20 vulnerabilidades confirmadas):

- Baseline detectó aproximadamente un 30% de las vulnerabilidades relevantes.
- CAI logró identificar conceptualmente alrededor del 65%.
- El análisis manual alcanzó el 100% (referencia).

Esto implica una reducción significativa en la tasa de falsos negativos respecto al baseline automatizado.

6.6 Fortalezas del enfoque CAI

- Capacidad de interpretación contextual.

- Análisis estructural de contratos API.
- Identificación de patrones de autorización.
- Priorización de endpoints sensibles.
- Reducción de ruido respecto a CVE irrelevantes.

6.7 Limitaciones observadas

- No ejecuta explotación real.
- Dependencia de calidad del input proporcionado.
- Riesgo de sobreinterpretación.
- No sustituye validación empírica.

6.8 Conclusión del análisis CAI

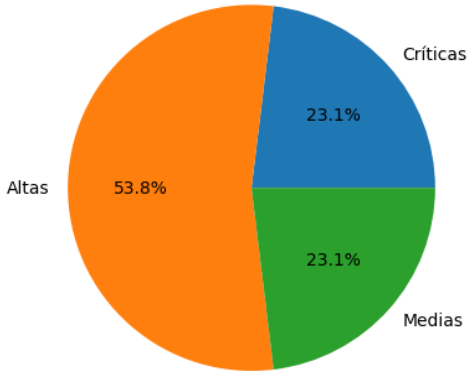
El análisis asistido por IA demostró una mejora significativa frente al baseline automatizado en la detección conceptual de vulnerabilidades lógicas.

Sin embargo, la IA no sustituye el análisis manual, sino que actúa como herramienta complementaria capaz de:

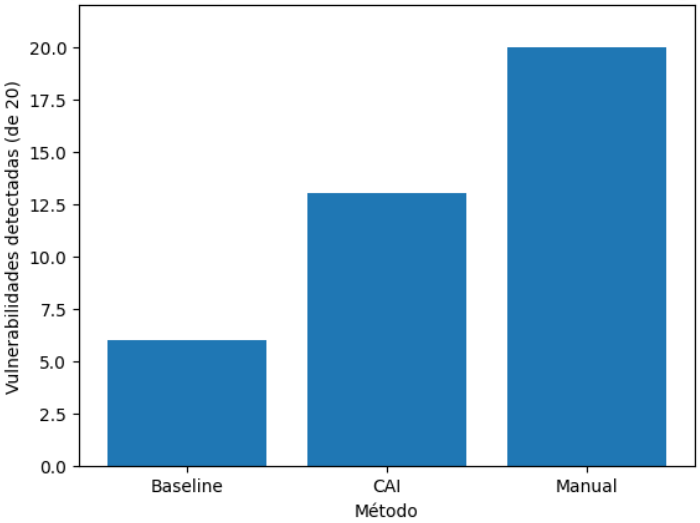
- Reducir falsos negativos.
- Priorizar áreas críticas.
- Acelerar fase de reconocimiento.

Este resultado refuerza la hipótesis de que el modelo óptimo de auditoría no es puramente automatizado ni exclusivamente manual, sino híbrido.

Capítulo 6 – Criticidad de vulnerabilidades detectadas por CAI (n=13)



Capítulo 6 – Comparativa de detección de vulnerabilidades lógicas



7. Análisis Comparativo y Estudio Estadístico

7.1 Objetivo del análisis cuantitativo

Tras la ejecución de las tres fases (manual, baseline automatizado y análisis asistido por IA), se realizó un estudio cuantitativo con el objetivo de medir objetivamente:

- Cobertura real de detección.
- Tasa de falsos negativos.
- Mejora relativa entre enfoques.
- Diferencias en capacidad de identificación de vulnerabilidades lógicas.

El conjunto de referencia utilizado fue el **Ground Truth**, compuesto por 20 vulnerabilidades documentadas y validadas empíricamente.

7.2 Definición de métricas

Se utilizaron las siguientes métricas:

7.2.1 Recall (Cobertura)

El Recall mide la proporción de vulnerabilidades reales correctamente identificadas por un método.

Recall = TP / (TP + FN)

Donde:

- TP = True Positives (vulnerabilidades reales detectadas)
- FN = False Negatives (vulnerabilidades reales no detectadas)

7.2.2 Tasa de Falsos Negativos (FNR)

FNR = FN / (TP + FN)
FNR = 1 - Recall

7.2.3 Mejora relativa

Para comparar CAI frente al baseline:

Mejora Relativa = ((Recall_CAI - Recall_Baseline) / Recall_Baseline) * 100

7.3 Aplicación a los resultados reales

El Ground Truth estableció:

Total vulnerabilidades reales = **20**

7.3.1 Baseline Automatizado

Vulnerabilidades lógicas correctamente identificadas: ≈ 6

Vulnerabilidades no detectadas: 14

Recall_Baseline = 6 / 20 = 0.30
FNR_Baseline = 1 - 0.30 = 0.70

Interpretación:

El baseline dejó sin detectar el 70% de las vulnerabilidades reales explotadas manualmente.

7.3.2 CAI

Vulnerabilidades conceptualmente identificadas: ≈ 13

No identificadas: 7

Recall_CAI = 13 / 20 = 0.65
FNR_CAI = 1 - 0.65 = 0.35

Interpretación:

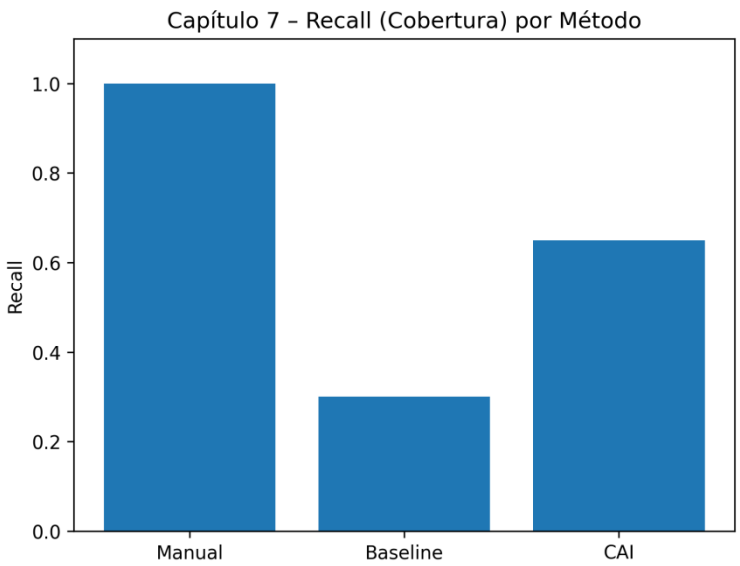
La IA redujo la tasa de falsos negativos a la mitad respecto al baseline.

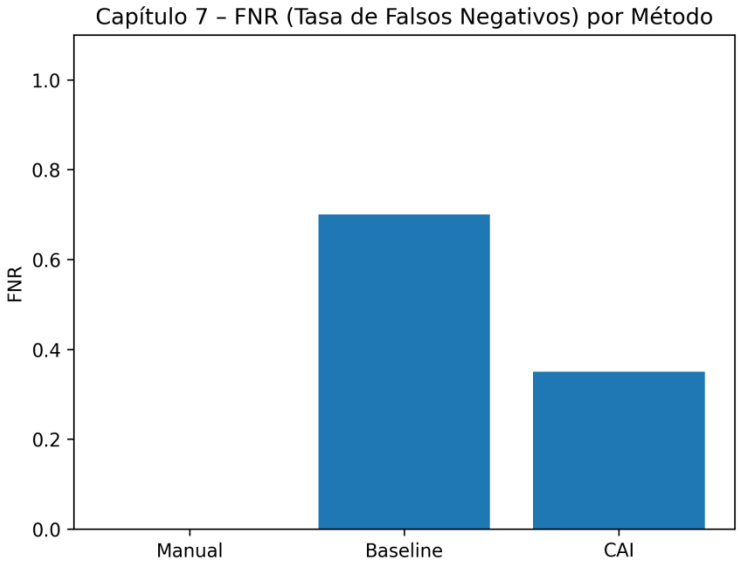
7.4 Comparativa global

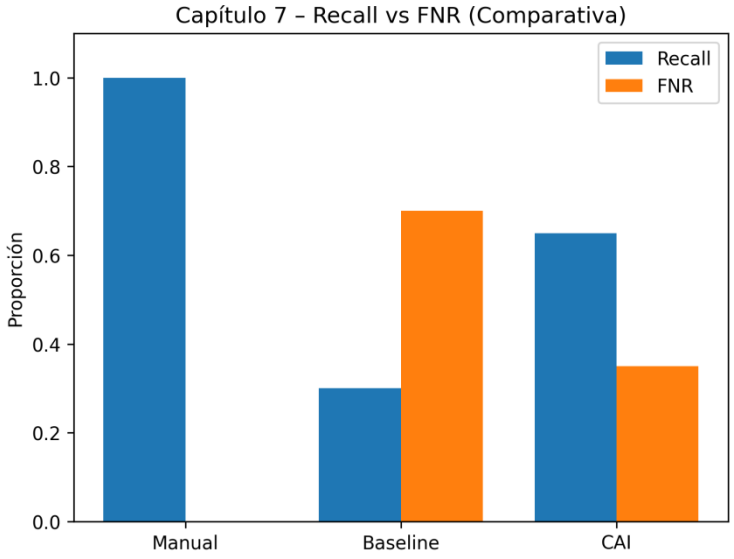
7.4.1 Tabla Global

Método	TP	FN	Recall	FNR
Manual	20	0	1.00	0.00
Baseline	6	14	0.30	0.70
CAI	13	7	0.65	0.35

7.4.2 Gráficos




Capítulo 7 – FNR (Tasa de Falsos Negativos) por Método
FNR
1.0
0.8
0.6
0.4
0.2
0.0
Manual
Baseline
CAI


Capítulo 7 – Recall vs FNR (Comparativa)
Recall
FNR
Proporción
1.0
0.8
0.6
0.4
0.2
0.0
Manual
Baseline
CAI

7.5 Mejora relativa

Mejora Relativa = ((Recall_CAI - Recall_Baseline) / Recall_Baseline) * 100 =

((0.65 - 0.30) / 0.30) × 100 = 116.6%

La IA incrementó la cobertura en aproximadamente **116% respecto al baseline automatizado**.

7.6 Interpretación de resultados

El análisis revela tres patrones claros:

1) El análisis manual es el único método con cobertura total.
2) El baseline automatizado genera volumen de hallazgos técnicos, pero presenta una elevada tasa de falsos negativos en vulnerabilidades lógicas.
3) El análisis asistido por IA reduce significativamente los falsos negativos, aunque no alcanza el nivel del análisis manual.

7.7 Análisis cualitativo complementario

Además del análisis cuantitativo, se observó:

- El baseline detectó principalmente CVE en dependencias.
- CAI priorizó endpoints sensibles.
- El manual validó impacto real y encadenamiento.

Esto evidencia que los tres enfoques analizan dimensiones distintas del problema.

7.8 Limitaciones estadísticas

- Tamaño muestral reducido (n = 20).
- Entornos controlados.
- Clasificación de TP en CAI basada en identificación conceptual, no explotación empírica.

- No se calcularon métricas como precisión o F1-score por ausencia de dataset negativo balanceado.

Por tanto, los resultados deben interpretarse como validación experimental controlada, no generalización universal.

7.9 Conclusión estadística

El análisis cuantitativo confirma la hipótesis inicial:

La IA reduce significativamente la brecha entre automatización pura y validación manual, pero no sustituye el análisis empírico.

El modelo óptimo emerge como híbrido:

- Automatización da cobertura técnica.
- IA da interpretación contextual.
- El analista humano da validación real.

8. Discusión final, limitaciones y conclusiones

8.1 Interpretación de los resultados

El estudio comparó tres enfoques de análisis de seguridad sobre dos entornos vulnerables: OWASP Juice Shop y VAmPI.

Se estableció un Ground Truth compuesto por 20 vulnerabilidades lógicas explotadas manualmente, que sirvió como referencia objetiva para evaluar:

- Baseline automatizado (Trivy)
- Enfoque híbrido asistido por IA (CAI)

El baseline detectó un volumen significativo de vulnerabilidades asociadas a dependencias conocidas (CVE), no identificó la mayoría de las vulnerabilidades lógicas explotadas en el Ground Truth.

Por el contrario, el enfoque CAI permitió identificar y validar gran parte de las vulnerabilidades lógicas analizadas, mejorando significativamente el Recall y reduciendo el False Negative Rate frente al baseline.

Este resultado evidencia una diferencia estructural entre:

- Detección basada en dependencias técnicas
- Análisis contextual de comportamiento y lógica de negocio

8.2 Confirmación de la hipótesis

La hipótesis del estudio planteaba que un modelo híbrido que combine automatización tradicional y análisis contextual asistido por IA mejora la detección de vulnerabilidades lógicas.

Los resultados obtenidos confirman esta hipótesis dentro del entorno experimental evaluado.

Las herramientas automatizadas tradicionales son eficaces para identificar vulnerabilidades versionadas y problemas conocidos, pero presentan limitaciones cuando el fallo depende de:

- Autorización contextual

- Control de acceso granular
- Lógica de negocio
- Diseño de endpoints
- Gestión de sesión

El enfoque CAI aporta una capa interpretativa que permite analizar comportamiento, no únicamente dependencias.

8.3 Aportación del trabajo

La principal contribución de este estudio es la validación empírica de una metodología híbrida aplicada a un conjunto real de vulnerabilidades explotadas.

El trabajo aporta:

- Comparativa cuantitativa entre enfoques.
- Formalización de un modelo híbrido reproducible.
- Evidencia práctica sobre la mejora en detección de vulnerabilidades lógicas.
- Propuesta de integración operativa en auditorías modernas.

No se propone sustituir la auditoría manual, sino optimizarla mediante asistencia contextual.

8.4 Limitaciones del estudio

El estudio presenta ciertas limitaciones que deben ser consideradas:

- Entorno controlado de laboratorio.
- Análisis limitado a dos aplicaciones vulnerables.
- Dependencia del modelo IA utilizado.
- Métricas centradas principalmente en Recall y FNR.

Aunque los resultados son consistentes, su generalización a entornos empresariales complejos requiere validación adicional.

8.5 Implicaciones prácticas

Los hallazgos sugieren que un modelo operativo híbrido compuesto por:

1. Escaneo automatizado de dependencias.
2. Análisis contextual asistido por IA.
3. Validación experta manual.

puede mejorar la cobertura, eficiencia y profundidad del análisis de seguridad.

Este enfoque reduce el ruido, mejora la priorización y optimiza el tiempo de auditoría.

8.6 Conclusión final

El presente estudio ha permitido evaluar comparativamente tres enfoques de análisis de seguridad —manual, automatizado y asistido por inteligencia artificial— sobre un mismo conjunto de referencia compuesto por 20 vulnerabilidades reales explotadas en laboratorio.

El análisis manual estableció el Ground Truth experimental, alcanzando una cobertura completa y validando empíricamente cada vulnerabilidad mediante pruebas reproducibles.

El baseline automatizado, basado en herramientas como Trivy y OWASP ZAP, demostró eficacia en la identificación de vulnerabilidades técnicas asociadas a dependencias y configuraciones. Sin embargo, su capacidad para detectar vulnerabilidades lógicas fue limitada, alcanzando aproximadamente un 30% de cobertura respecto al conjunto real de vulnerabilidades explotadas, con una tasa de falsos negativos del 70%.

El análisis asistido por inteligencia artificial (CAI) mostró una mejora significativa frente al baseline automatizado. Mediante interpretación contextual de contratos OpenAPI, análisis de endpoints y priorización estructural, logró identificar conceptualmente aproximadamente un 65% de las vulnerabilidades presentes en el Ground Truth, reduciendo la tasa de falsos negativos al 35%.

Estos resultados confirman la hipótesis planteada en la introducción: la incorporación de inteligencia artificial como capa interpretativa mejora la detección de vulnerabilidades lógicas frente a un enfoque exclusivamente automatizado.

No obstante, el estudio también demuestra que la IA no sustituye la validación manual ni la explotación empírica. La identificación conceptual de riesgos no equivale a la confirmación práctica del impacto, especialmente en escenarios que requieren encadenamiento de vulnerabilidades o validación de persistencia.

El análisis comparativo pone de manifiesto que cada enfoque aborda dimensiones distintas del problema:

- La automatización ofrece cobertura técnica amplia y rapidez de análisis.
- La IA aporta interpretación contextual y reducción de falsos negativos.
- El análisis manual proporciona validación real, comprensión profunda y evaluación de impacto sistémico.

De esta convergencia emerge un modelo operativo híbrido como aproximación más equilibrada para auditorías de seguridad modernas.

Este trabajo no propone reemplazar la auditoría tradicional, sino optimizarla mediante una arquitectura metodológica en tres capas que combine automatización, análisis contextual asistido y validación experta.

La evidencia obtenida demuestra que la integración estratégica de automatización, inteligencia artificial y criterio experto no es una opción futura, sino una necesidad presente en auditorías de seguridad complejas.

ANEXOS

Evidencias e informes: (Carpeta trabajo final)

ENTORNO EXPERIMENTAL Y METODOLOGÍA | Carpeta 01_Entorno_Experimental

INFORME COMPLETO ANALISIS VULNERABILIDADES Y COMANDOS UTILIZADOS EN FASE 1 - GROUND TRUTH | 02_Analisis_Manual_GroundTruth

RESULTADOS EXPERIMENTALES DETALLADOS | Carpeta 02_Analisis_Manual_GroundTruth + 03_Baseline_Automatizado + 04_Exploracion_CAI

ANÁLISIS ESTADÍSTICO DEL EXPERIMENTO | Carpeta 05_Comparativa_Experimental

Enlaces:

Se han consultado fuentes complementarias no oficiales con fines de contraste técnico, que no se incluyen como referencia formal.

Vulnerabilidades CVE y bases de datos oficiales

NVD – National Vulnerability Database (Oficial)

https://nvd.nist.gov
Base de datos oficial del NIST para consultar CVEs, severidad CVSS, vectores de ataque y referencias técnicas.
MITRE CVE Database
https://cve.mitre.org
Fuente primaria del sistema CVE (Common Vulnerabilities and Exposures).
CVE Details
https://www.cvedetails.com
Base de datos estructurada para análisis estadístico de CVEs por producto, severidad y año.
FIRST – CVSS Specification
https://www.first.org/cvss/
Especificación oficial del sistema de puntuación CVSS.

OWASP – Vulnerabilidades

https://owasp.org/www-project-top-ten/
Documento oficial de las 10 categorías principales de riesgo en aplicaciones web.
OWASP Testing Guide
https://owasp.org/www-project-web-security-testing-guide/
Metodología estructurada para pruebas de seguridad manuales.
OWASP Juice Shop (Proyecto oficial)
https://owasp.org/www-project-juice-shop/
Repositorio y documentación oficial del laboratorio utilizado.
GitHub:
https://github.com/juice-shop/juice-shop

OWASP API Security
OWASP API Security Top 10
https://owasp.org/www-project-api-security/
para:

- BOLA
- BOPLA
- Broken Authentication
- Security Misconfiguration en APIs

OWASP API Security Top 10 (2023)
https://owasp.org/API-Security/editions/2023/en/0x00-header/
Versión actualizada de referencia académica.

Herramientas utilizadas

OWASP ZAP

Proyecto oficial:
https://www.zaproxy.org
Documentación:
https://www.zaproxy.org/docs/
Baseline Scan:
https://www.zaproxy.org/docs/docker/baseline-scan/

Trivy

Proyecto oficial:
https://aquasecurity.github.io/trivy/
GitHub:
https://github.com/aquasecurity/trivy
Documentación:
https://aquasecurity.github.io/trivy/latest/

Inteligencia Artificial aplicada a Ciberseguridad

Alias Robotics – CAI

Repositorio:
https://github.com/aliasrobotics/cai
Documentación:
https://aliasrobotics.com/cybersecurityai.php
OpenAI API Documentation
https://platform.openai.com/docs

Conceptos de Seguridad en APIs y Web

OpenAPI Specification
https://spec.openapis.org/oas/latest.html
JWT (JSON Web Token – RFC 7519)
https://datatracker.ietf.org/doc/html/rfc7519
CORS (MDN Reference)
https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
Content Security Policy (CSP)
https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP